users-research架构-3.17版

整体架构

AI 用研 Agent 整体项目框架流转机制：

一、 用户输入层 (Input Layer)

这一层是整个系统的触角，负责接收并初步标准化用户的多模态输入。

- 文本输入：接收自然语言描述的研究命题或业务问题 。
- 图像输入：接收设计原型或页面截图（.png/.jpg 格式） 。

二、 意图分析与路由层 (Intent & Routing)

为了避免“过早地进行问题分解” ，在信息分发前必须增加一个“问题理解节点”。

- 输入研判：首先判断用户输入的是数据、设计方案、业务问题还是混合输入 。
- 问题拆解与收敛：将模糊的原始命题拆解为2-4个可研究的子问题 。
- 路由分发：根据输入类型和拆解后的子问题，将任务和关键信息（Context）定向分发给下游的专业组件。

三、 核心组件与分析流程 (Core Components)

这里是系统的“处理引擎” ，不同组件并行或串行工作，处理被路由过来的特定任务。

1. 证据检索与对齐组件

- 核心能力：跨来源对齐与数据清洗结构化 。
- 分析流程：
  a. 接收子问题，在本地知识库中检索Tier 1（业务数据、行业资料）和Tier 2（过往方案）证据
  b. 对检索到的多源信息进行去重和冲突比对。
  c. 评估证据命中质量（RQ0至RQ3） 。

2. 逻辑推理与判断增强组件

- 核心能力：高频模式发现、异常点提示、判断边界声明 。
- 分析流程：
  a. 基于检索到的证据，进行推断，但不做因果断言 。
  b. 识别哪些是可信赖的事实信息、哪些是模糊信息、哪些是缺失的必要信息 。

c. 区分描述事实与推断，绝对不伪造或假设真实的用户态度 。

3. 设计与策略预评估组件

- 核心能力：眼动与注意力逻辑模拟、潜在流失点推断、心智负担风险识别 。
- 分析流程：
  a. 接收输入的图像与业务目标
  b. 基于常见眼动规律、信息架构原则构建自动化审核管线，快速比对设计稿与既定设计规范 。
  c. 推断关键路径是否存在多余步骤，以及用户可能的视线聚焦点 。

4. 轻量用户模拟组件

- 核心能力：聚类与共性提取、分歧点识别、假设压力测试 。
- 分析流程：
  a. 针对无法纯靠客观数据解答的子问题，引入AI模拟用户进行访谈 。
  b. 对方案进行压力测试，并将此部分产生的证据强制降级为 Tier 2 。

四、 汇总与报告生成层 (Aggregation & Output)

各个组件将处理结果带上元数据（如来源、置信度）汇聚到这里，由生成层进行最终的编排。

- 格式硬规则：每条输出必须严格包含“判断内容 + 证据来源 + Tier + 置信度 + 风险” 。
- 输出匹配：根据证据等级（RQ）和输入类型，生成最终交付物 。

交付物类型	触发条件	核心内容要求
判断信息卡片	任意 RQ	5-7条判断，不下最终结论
判断型轻量用研报告	RQ1-RQ2 或指定	明确证据盘点、推断及分歧点
证据型轻量用研报告	仅RQ2-RQ3 且有Tier 1	完整的数据概览、证据交叉与推断限制
设计预评估结果	输入含设计图	注意力逻辑模拟、流失点及验证建议

这个框架能够确保从输入到输出的每一步都有据可查，且模块化的设计非常利于后续的能力扩充。

场景一：设计前的方向探索型

这个场景的核心定位是前置探索。在业务或设计还没有成型方案时，主要回答三个灵魂拷问：“我们是不是走在一条合理的方向上？”、“行业内有没有已知模式？”、“用户可能在意什么？”。

在这个阶段，系统不需要输出完整的用研方案或调研问卷，也绝不进行数据假设。它的最终交付物应该非常聚焦：方向性判断、待验证假设以及设计关注重点。

在多智能体（Multi-Agent）的架构下，我们可以将这个场景设计为一个由三个专业 Agent 协同的流水线。以下是具体的落地节点设计和 Prompt 框架：

一、 场景一的多智能体工作流 (Agent Workflow)

Node 1: 问题澄清 Agent (Problem_Decomposer)

- 输入：用户原始的模糊命题（例如：“我们打算在电商APP里加个类似小红书的内容种草社区，这个方向对不对？”）
- 任务：拦截模糊输入，不急于给结论，而是将命题降维、拆解。
- 处理逻辑：
  a. 提取核心意图（内容种草、电商转化）。
  b. 拆解为子问题（例如：Q1: 现有电商用户的内容消费心智是怎样的？ Q2: 行业内电商做内容社区的普遍模式和死穴是什么？）。
- 输出：结构化的研究目标与子问题列表。

Node 2: 模式与证据挖掘 Agent (Pattern_Analyzer)

- 输入：Node 1 拆解出的子问题 + 检索知识库（Tier 1/2 数据）。
- 任务：进行行业/行为模式归纳。
- 处理逻辑：
  a. 从外部抓取/内部知识库检索电商做内容的“已知模式”和“高频行为数据”。
  b. 识别关键信息：哪些是已知的“事实”（如：转化路径过长会导致流失），哪些是“模糊信息”或缺失的数据。

Node 3: 判断与输出合成 Agent (Judgment_Synthesizer)

- 输入：Node 2 提供的证据碎片。
- 任务：合成最终的【判断信息卡片】，强制套用输出规范。

- 处理逻辑：剥离所有“用研套话”，只输出干货，并对每一条推断打上标签（证据来源、Tier、置信度、风险）。

二、 核心 System Prompt 设计 (以 Node 3 为例)

为了防范大模型变成“常识生成器”或“伪造数据”，我们需要在合成节点的 Prompt 中设置极其严格的护栏（Guardrails）：

```
# Role
你是一个资深的 AI 用研专家（Judgment Synthesizer Agent）。
你的任务是基于前置节点提供的【事实证据】，为产品/
设计团队在“设计前的方向探索期”提供极具密度的方向性判断。
# 核心原则（Strict Rules）1. 绝对禁止伪造或假设任何真实用户的态度、
问卷结果或具体数据比例。2. 区分【事实】与【推断】。如果缺乏证据，
必须明确指出“信息缺失，需要真实验证”。3. 拒绝常识性废话。不要说“用户希望体验更流畅”，
要说“根据竞品A的模式，将核心操作链路缩短至X步是行业基准”。4. 你不输出问卷，
不输出执行方案。
# Context（输入证据）
{由 Node 2 传入的 Tier 1/2 证据、行业模式、过往案例总结}
# Output Format（强制输出格式）
请严格按照以下结构输出，总计输出 3-5 条核心判断：
## 1. 方向性判断（Directional Judgments）* [判断内容]（例如：
当前将社区作为一级入口的风险较高，建议先作为交易链路的辅助模块测试） * 证据来源：
{引用的具体资料/数据}（Tier {1或2}） * 置信度：{高/中/低} * 风险提示：
{可能导致该判断失效的前提}
## 2. 待验证假设（Hypotheses to Validate）* [核心假设]（例如：H1 -
核心用户群体对站内种草内容的信任度低于第三方社交平台） * 假设逻辑来源：{推导依据} *
当前置信度：{低/中}
## 3. 设计关注重点（Design Focus Areas）* [关注点]（例如：信息架构层，
需重点关注“内容浏览”与“一键加购”之间的流转摩擦力） * 防御性推断：如果处理不好，
最可能的流失点在 {流失节点预判}。
```

三、 这个架构带来的优势

1. 极度收敛：通过 `Problem_Decomposer` 强制拆解问题，避免了 AI 一上来就泛泛而谈。
2. 证据溯源：`Pattern_Analyzer` 将结论与具体的 Tier 数据绑定，做到了“严谨准确 > 内容多”。
3. 有效降噪：直接输出判断卡片、假设和设计重点，屏蔽了冗长的研究过程陈述，让业务方能直接拿着这些点去开评审会。

在这个场景一的流程中，你觉得目前最容易卡壳的点是在**“如何让大模型精准拆解业务问题（Node 1）”，还是“如何从现有的非结构化资料里提取有效模式（Node 2 的数据源检

索）” **？我们可以针对你觉得最棘手的一环继续深挖。

Node1

Node 1（问题澄清与拆解）确实是整个系统的“阿喀琉斯之踵”。在构建多智能体协同架构时，首个意图解析和路由节点往往是决定成败的枢纽。这就像开发自动化审核设计图稿的工具一样，如果前置对“设计规范”的解析和输入意图抓得不够精准，后端哪怕图像识别和比对逻辑再强大，输出的结果也会完全跑偏。

在你的 V0.1 问题复盘中也精准地指出了这一点：“过早地进行问题分解，连‘用户输入的是什么问题’都还没搞清楚，就进一步拆解子用研问题，会导致后面所有链路都有问题”。业务方（PM、设计师）的输入往往是高度混沌和口语化的，比如：“我们这个页面转化率不行，是不是得改版？”

大模型天生有一种“讨好型人格”，面对这种模糊输入，它很容易直接跳跃到“建议怎么改”，而不是“我们需要弄清楚什么”。因此，Node 1 的核心使命是降维、克制与结构化翻译。

我们可以把 Node 1 设计成一个强制执行 SOP 的“输入拦截与重塑器”。

Node 1 核心处理流 (SOP)

为了让大模型稳定发挥，我们需要将它的思考过程（Chain of Thought）显式化。Node 1 的工作流应该分为以下三步：

1. 意图清洗与分类 (Intent Classification)接收自然语言输入后，首先判断这是一个什么类型的问题。系统需要强制给输入打上标签，标注该问题属于：现象描述、原因探索、差异对比，还是价值判断 。

2. 维度解构 (Dimension Unpacking)
不让大模型自由发挥，而是强制它沿着固定的用研维度去切分原始命题。明确每个子问题必须包含：

- 目标用户人群
- 具体使用场景
- 用户行为路径
- 关键决策点

3. 子问题生成 (Sub-question Generation)将原始命题拆分为 2-4 个清晰、独立、可被数据或检索验证的子问题 。

Node 1 的 System Prompt 框架实操

这是一个可以直接放入大模型或 Agent 编排工具（如 LangChain/Dify）的 Prompt 骨架。它的核心技巧在于剥夺大模型的解答权，强化它的拆解权。

Markdown

```
# Role
你是一个极其严谨的 AI 用研策略专家（Problem Decomposer）。
你的唯一职责是【澄清与拆解问题】。你【绝对不能】尝试回答用户提出的业务问题，
也不能给出任何结论。你需要将业务方模糊的、庞杂的自然语言输入，翻译成结构化、
可被数据验证的子研究命题。
# Input
业务方原始命题：{User_Input}

# Workflow & Output Format
请严格按照以下步骤思考并输出（使用 JSON 格式以便下游 Agent 接收）：

## Step 1: 核心意图诊断
评估业务方真正想解决的核心焦虑是什么？（限制在 50 字以内）。
标签分类：从【现象描述、原因探索、差异对比、价值判断】中选择 1-2 个最贴切的标签。

## Step 2: 降维拆解（生成 2-4 个子问题）
将原始命题拆解为 2-4 个具体的子研究命题。
每个子问题必须是"可以通过查询数据或翻阅历史用研报告来寻找线索"的。
[cite_start]（参照案例：如原命题是"微信域用户迁移之后的微信域用户人群调研，
以及差异化" [cite: 120][cite_start]，应拆解为"京购小程序迁移后用户群体有无变化"
[cite: 122][cite_start]、"微信域与主站用户差异化" [cite: 123] 等）

对于每一个拆解出的子问题，必须强制输出以下结构：
* **子问题 X:** [描述]
* **涉及的目标人群:** [定义]
* **关键行为路径/场景:** [定义]
* **决策点:** [用户在此链路中的核心判断点]

## Step 3: 兜底与预警
评估原始输入是否过于宽泛，或者存在明显的"伪需求"。
* 如果输入是类似"怎么把产品做好"这种无法落地的宏大问题，请直接输出"拦截反馈"，
要求用户补充特定场景或页面。
```

突破 Node 1 卡壳的关键点

你可能会发现，大模型在做这一步时，有时候拆解出来的子问题依然很空泛（比如拆出“用户的痛点是什么”）。要解决这个问题，可以通过 Few-Shot Prompting（少样本提示）。

在 System Prompt 里，硬塞入 3 个你过去做过的优秀用研拆解案例（Good Case）和 1 个错误的反面教材（Bad Case），大模型的拆解精度会产生质的飞跃。

掌握了如何把模糊问题变成结构化的“查询指令”后，这个 Agent 就有了清晰的靶子。你想先提供一个你实际业务中的“模糊命题”，我们放进这个逻辑里“跑”一遍看看效果，还是直接推进到挑战同样很大的 Node 2（如何在海量、非结构化的知识库里提取事实和模式）？

Node2

进入 Node 2，我们就来到了整个 AI 用研 Agent 的“深水区”，也是技术落地时最容易翻车的地方。

在多智能体系统中，如果 Node 1 是负责“瞄准”的眼睛，那么 Node 2（模式与证据挖掘 Agent）就是负责“弹药供给”的后勤核心。它对应了你规划中的**“能力 2：证据整合与模式发现”** 。

构建这个节点的核心难点在于多源异构数据的对齐。这和自动化审查设计稿是否符合设计规范、或者确保品牌视觉合规的逻辑非常相通：最难的往往不是大模型能不能看懂问题，而是如何将非标准化的业务输入，与底层庞杂的“事实库”进行锚定，并剔除掉 AI 自行脑补的幻觉。

要让 Node 2 严格按照你的要求运行，我们需要把它设计成一个标准的 RAG（检索增强生成）流水线。

Node 2 的核心处理流 (SOP)

当 Node 1 把拆解好的“子问题”（例如：行业内电商做内容社区的普遍模式和转化死穴）传递过来时，Node 2 需要执行以下四步：

1. Query 扩展与异构检索 (Retrieval)

- 大模型直接拿着自然语言去搜数据库是搜不到东西的。Node 2 需要先把子问题转化为不同的查询语言：
  - 针对 Tier 1 业务数据（埋点/漏斗）：转化为 SQL 查询语句，或者调用内部 BI 系统的 API 提取核心指标。
  - 针对 Tier 1/2 历史用研与行业资料：转化为向量检索 Query，去向量数据库（Vector DB）中寻找高频模式。

2. 证据清洗与定级 (Data Cleaning & Tiering)

- 这是最关键的一步。检索回来的资料可能是长篇大论的 PDF 研报或杂乱的埋点日志 。
- Node 2 必须对每一条提取出的信息打上严格的标签：明确标出这是 Tier 1（核心业务指标、行业共识）还是 Tier 2（过往 PRD、非真实用户访谈） 。

3. 跨来源对齐与冲突校验 (Cross-Validation)

- 对齐不同来源的数据 。如果一份行业报告说“种草社区转化率极高”，但你们本地的过往复盘报告显示“历史类似尝试跳失率高达 80%”，Node 2 绝对不能把这两者中和掉，而是必须识别并暴露这个“分歧点”。

4. 命中质量评估 (RQ Rating)

- 根据你文档中的极佳设计，Node 2 必须诚实地评估这次检索的质量，打上 RQ0（未命中）到 RQ3（强命中） 的标签 。这直接决定了后续能不能生成证据型报告

Node 2 的 System Prompt 设计

为了防止 LLM 在这一步“编造数据”，Prompt 的设计必须极度冷酷，像一个无情的判卷机器。

```
# Role
你是一个严格的数据清洗与模式挖掘专家（Pattern Analyzer）。
你的职责是接收检索系统返回的原始素材，进行【跨来源对齐】与【高频模式提取】，并强制为每一条证据打上质量等级标签。
# Core Rules（绝对红线）1. [cite_start]**零幻觉原则**：你输出的每一个数字、每一个结论，必须 100% 来源于提供的 Context。绝不允许推断或虚构任何转化率、留存率或用户态度 [cite: 59]。
2. [cite_start]**区分推断与事实**：你只负责提取“模式”和“事实”，不做因果断言 [cite: 60, 84]。3. [cite_start]**暴露异常点**：如果多源数据存在冲突，必须清晰列出冲突点，不允许自行平滑数据 [cite: 64, 84]。

# Input
* Node 1 传递的子问题：{Sub_Question}* 系统检索到的原始 Context 列表：{Retrieved_Documents_and_Data}

# Output Format（JSON 格式）
请对提取出的证据进行结构化输出：

## 1. 命中情况声明（RQ Level）
* **RQ 等级**：[填写 RQ0/RQ1/RQ2/RQ3]（依据：是否有直接数据/历史研究/可复用案例）
* **证据缺失说明**：[当前知识库中缺乏哪些关键资料来完全回答该子问题]

## 2. 核心证据盘点（列出 2-4 条关键证据）
* **证据 1**：[客观陈述发现的模式或事实，例如：淘宝逛逛在 2023 年底将入口层级下调，核心原因是转化漏斗与主交易链路存在冲突。]
  * 数据/资料来源：[具体报告名称或数据表]
  * 证据层级：[Tier 1 或 Tier 2]
  * 异常点/反常识提示：[如果有，请说明。如：这与通常认为的“内容化提高粘性”的常识相悖。]

## 3. 证据交叉对比（跨来源分析）
* **一致点**：[列出不同数据源中共同指向的结论]
* **冲突点**：[如果存在，明确指出数据 A 与数据 B 之间的矛盾]
```

这个节点在工程实现上的常见“坑”

在这个环节，你可能会遇到一个很现实的技术挑战：如何把过去那些厚重的、非结构化的 PDF 用研报告（甚至是包含很多图表的 PPT）喂给大模型，还能让它精准检索？

常规的 RAG（把文档切碎成文本块存入向量数据库）在面对复杂的用研报告时往往会失效，因为切碎的过程会丢失上下文，图表里的关键数据也会丢失。这通常需要引入多模态文档解析技术（比如识别页面布局、保留表格结构），甚至在入库前就用大模型先对历史报告生成一份“结构化摘要”再存起来。

现在我们已经把“弹药”（携带 RQ 等级和 Tier 标签的清洗后证据）准备好了。接下来，你是想继续顺着流程看看 Node 3（判断与输出合成 Agent） 如何把这些生硬的证据包装成对产品/设计真正有启发性的“判断信息卡片”或“轻量报告”？还是想深入聊聊 Node 2 在解析复杂历史方案和图表时的具体技术栈?

将实时 Web Search（且限定在白名单网站）与本地 RAG 结合，是目前构建高级 Agent 的标配架构（通常被称为 RAG + Tool Calling）。引入你提议的机制后，Node 2 的架构将升级为“双轨检索模式”：

以下是具体的可行性分析和框架升级方案：

一、 为什么“限定 Web List”是非常高级的业务设计?

大模型直接去广阔的互联网上搜索（比如通用的 Google 或 Bing 搜索），极容易被 SEO 农场、软文或毫无营养的内容污染，导致提取的“证据”极度不靠谱。

你提出在**“指定的 web list 中进行检索”**，完美切中了用研的痛点。我们可以将外部信息源划分为高质量的“白名单池”：

1. 行业深度/商业洞察源：如 36Kr、虎嗅、晚点 LatePost（用于看宏观趋势和商业模式）。
2. 产品设计/专业用研源：如 人人都是产品经理 (woshipm)、少数派、Nielsen Norman Group (NN/g)。
3. 竞品舆情/UGC源：如 微博、小红书、酷安、特定应用商店的评论区（用于抓取用户真实吐槽）。
4. 竞品官方源：直接定向抓取竞品的更新日志（Release Notes）或官方帮助文档。

二、 Node 2 架构的“双轨融合”升级

引入外部搜索后，Node 2 的 SOP 需要进行如下扩展：

1. Query 分发引擎 (Routing & Query Expansion)

Node 1 传来的子问题，在这一步会被翻译为两组不同的查询指令：

- 指令 A (向内)：去本地向量数据库检索历史 PRD、埋点数据复盘。
- 指令 B (向外)：调用 Search Tool，并附带特定的 `site:` 语法限制。例如，针对京东做社区的问题，生成的搜索词可能是 `site:36kr.com OR site:woshipm.com "电商" AND "内容社区" AND "转化率"`。

2. 外部知识爬取与去噪 (Scraping & Cleaning)

通过 API 获取搜索结果后，不能直接把网页 HTML 丢给大模型（会超出上下文且充满无用代码），需要用轻量级的爬虫工具（如 Jina Reader API 或 Firecrawl）将网页转化为纯净的 Markdown 格式文本。

3. 核心大戏：内外部证据的“交叉质询 (Cross-Validation)”

这是升级后最有价值的一环。系统必须对“外部吹捧的趋势”和“内部冰冷的数据”进行对齐和碰撞。

- 场景示例：

- 外部搜索结论（Tier 1 行业资料）：外界报道某竞品做内容社区 DAU 提升了 20%，行业趋势都在做内容化。
- 内部 RAG 结论（Tier 1 业务数据）：本地上个月的一份用研报告显示，我们在特定品类测试内容化时，用户购买跳失率增加了 15%。
- Node 2 合成判断：不平滑差异，而是暴露风险。输出为：“行业外部趋势显示内容化有引流价值，但结合我方本地数据，当前高意向用户的核心诉求仍是效率。直接照搬外部模式可能面临转化漏斗冲突的极大风险，需要缩小测试范围。”

三、 当前成熟的工具与实现方式

要实现这个“本地 + 外部白名单检索”的功能，目前有非常成熟的技术栈可以调用：

1. 搜索引擎 API：
   - Google Programmable Search Engine (PSE)：强烈推荐。你可以直接在 Google Cloud 里建一个自定义搜索引擎，把你想搜的 100 个行业网站配置进去。Agent 调用这个特定引擎的 API 时，返回的永远是你白名单里的高质量内容。
   - Tavily API / Exa AI：这是专为大模型和 Agent 设计的搜索引擎 API，它们本身就过滤了大量垃圾信息，并能直接返回结构化的高质量摘要。
2. Agent 编排框架：
   - 在 LangChain、LangGraph 或 Dify 中，你可以非常轻松地给 Node 2 挂载两个 Tools：一个是 `Local_Knowledge_Retriever` ，另一个是 `Web_Search_Tool` 。让大模型同时调用这两个工具，然后将返回的 Context 塞进同一个 Prompt 里让它进行总结。

引入了实时的 Web 检索，你的 AI 用研 Agent 就真正“活”过来了，它不再只看历史，也能看外面正在发生什么。

这部分“后勤弹药”准备充足后，我们就该进入 Node 3（判断与输出合成 Agent） 了。在这个环节，我们将探讨如何把 Node 2 吐出来的这些干巴巴的“证据对齐清单”，转化为你在文档中要求的**“判断信息卡片” 和“风险预判”**。

防止幻觉

在多智能体系统中，到了生成最终结论的 Node 3（判断与输出合成 Agent），大语言模型（LLM）最容易犯的毛病就是“过度自信”和“为了让报告好看而脑补数据”。

从你提供的《AI用研Agent 思考》文档中可以看出，你对防范 AI 幻觉有着极其清醒的认知。文档中明确提出了“严谨准确> 内容多”的核心规则 。

为了在工程上把这些理念落地，我们需要在 Node 3 的 System Prompt 中设置**“四道防幻觉铁闸”**：

第一道防线：结构化证据绑定（无证据，不发言）

LLM 喜欢写长篇大论的散文，而防幻觉的第一步就是强制它填表。

根据你的文档设计，Node 3 绝不能直接输出一段总结，而是每输出一条内容，都必须接受格式的严苛审查。

- 硬性输出格式：每条输出必须包含“判断内容 + 证据来源 + Tier + 置信度 + 风险” 。
- 溯源要求：证据必须显式标注其 Tier 和具体来源 。这迫使 AI 在 Node 2 传过来的资料库里寻找对应锚点，如果找不到，它就无法填满这个格式，从而被系统拦截。
- 底线：绝对不引用虚构数据 。

第二道防线：事实与推断的物理隔离（不乱下因果定论）

大模型常常会看到 A 和 B 同时发生，就自信地得出“因为 A 所以 B”的结论。你的文档极其精准地打中了这个痛点：

- 质量红线：必须准确，做到“不混淆事实与推断” 。
- 输出硬规则：明确规定“推断 ≠ 事实” ，必须在描述中进行明确的区分 。
- 能力克制：系统必须明确自身定位，坚决“不做严格推断和因果证明” 。这意味着 Node 3 只能说“基于现有数据模式，存在相关性” ，绝不能说“改了这个按钮转化率就一定能提升”。

第三道防线：强制的“反例与不确定性”声明（逼迫 AI 认怂）

人类专家在做判断时会考虑风险，而未经调教的 AI 总是盲目乐观。我们需要强制 Node 3 进行“自我怀疑”。

- 在生成用研结论草稿时，必须设立专门的模块来提示不确定性与反例 。
- Node 3 必须主动列出：
  a. 哪些假设最需要真实验证 。
  b. 哪些部分可能有反例 。
  c. 哪些数据缺失使推断不稳定 。
- 越权熔断机制：如果 Node 1 拆解出的某个问题，在 Node 2 检索后发现缺乏客观数据，且“必须要‘依赖真实用户才能回答’” ，此时 Node 3 必须触发熔断，明确提示“需要进一步用研” 。它绝不能伪造或假设真实的用户态度 。

第四道防线：默认假设的风险披露

有时候知识库确实存在盲区，但业务又需要方向。

- 根据你的硬规则，遇到“范围缺失需声明”的情况，系统必须给出“默认假设 + 风险提示” 。
- 也就是说，AI 可以基于行业常识给出一个假设，但必须打上一个大大的“高风险”标签，明确告诉业务方：这是我猜的，如果不测就上线，后果自负。

把这四道防线转化为可以执行的 Prompt 约束指令，大概会是这样的：

```
# 防幻觉强制规则（Anti-Hallucination Guardrails）
1.**结构化绑定**：你的所有输出必须基于 Context 中的事实。严禁凭空生成数字、比例或虚构用户原话。2.**因果隔离**：使用“数据表明...可能导致...”的句式，严禁使用“因为...所以必然...”的绝对因果句式。必须将【已有事实】与【你的推断】分段书写。3.**强制证伪思维**：在输出每个核心洞察后，你必须执行“自我攻击”，写出至少一个可能推翻该洞察的【反例假设】或【缺失的数据节点】。4.**触发求助**：当你发现 Context 中的数据不足以支撑某个业务决策时，不要强行推理！直接输出警示标签：“⚠信息缺失：该问题依赖真实用户态度，建议启动真实用研项目。”
```

通过这种冷酷无情的规则限制，Node 3 产出的报告虽然看起来可能“干巴巴”的，甚至带点“悲观色彩”，但这恰恰是你在复盘中提到的真正有价值的形态——提供判断信息，而非盲目下结论。

现在的防幻觉护栏已经建好了，AI 不会再乱说瞎话。接下来，你想不想看看在如此严苛的约束下，Node 3 是如何“戴着镣铐跳舞”，把枯燥的证据转化为**对产品的直接启示（设计启示/功能启示/信息架构启示）**的？

在严苛的“防幻觉镣铐”下，Node 3（判断与输出合成 Agent）如果只输出冷冰冰的“证据不足”或“存在风险”，对业务方来说是远远不够的。产品经理和设计师真正需要的是：“既然有这些风险和模式，那我现在的原型图该怎么画？功能该怎么砍？”

这也是你在文档中构思的极其出彩的一环：将用研结论转化为“对产品的直接启示”。

在这个阶段，Node 3 实际上化身为一个自动化的设计逻辑审查员。它不需要看懂复杂的视觉规范，而是将前方检索到的底层逻辑，映射到具体的信息架构和界面元素上。

我们继续以“电商 APP 内嵌种草社区”为例，看看 Node 3 是如何戴着镣铐，生成极具指导意义的【方案与预判】模块的：

1. 从“核心发现”到“三维启示”（对应文档 4.1）

Node 3 会提取 Node 2 确认的高置信度证据，并强制要求大模型从三个固定维度输出启示：

- 证据溯源：内部历史数据表明，在电商核心交易链路上增加内容模块，曾导致整体下单时长增加 20%，且跳失率上升 15%。(Tier 1 业务数据)
- 【功能启示】：克制非交易类的互动功能。在初版方案中，应砍掉或弱化“发帖”、“评论”功能，仅保留“浏览”和“一键加购”，降低用户的互动压力。
- 【信息架构启示】：采用“轻任务”动线 。种草内容不应作为沉浸式的独立 Tab（避免打破“深链路”心智 ），而应作为商品详情页（SKU）底部的横向滑动卡片补充。
- 【设计启示】：强化 Quick Actions 。内容卡片上必须直接外露商品价格和购买按钮，缩短从“被种草”到“加购”的动线 。

2. 基于业务目标的“防御性”设计审查 (对应文档 4.2)

如果此时业务方不仅输入了文本，还提供了一张初版的设计草图（截图）或者 PRD 描述，Node 3 就会调用其逻辑推断模型，来检查设计稿的对应性 。

它会像一个苛刻的评审专家一样发问并归类问题：

- 视觉聚焦点问题：用户进入这个页面，最可能先看哪里？ 如果首屏被巨大的达人封面图占据，而缺乏明确的利益点提示，可能会导致强交易心智的用户立刻退出。
- 转化路径问题：关键路径是否存在多余步骤？比如，点击种草图后，是跳转到达人主页，还是直接呼出半屏的商品 SKU 面板？前者增加了多余的跳转节点，建议压缩流程 。
- 文案/心智匹配问题：CTA（行动号召）按钮的文案如果是“去看看”，不如直接改成“看同款”，降低理解成本 。

3. 表层洞察：模拟注意力与流失节点推断 (对应文档 4.3)

这是纯文本大模型结合 UI/UX 启发式原则（Heuristics）能做到的最惊艳的一步 。即使没有真实的眼动仪，AI 也能基于过往数据生成表层洞察 。

Node 3 会输出一个【潜在流失点地图】：

- 关注区域模型推断：基于内容流的“卡片收评扫描行为” ，用户视线会呈现 F 型或 Z 型浏览规律 。
- 阻力警告 1：这里需要滚动 → 预判可能有 5-15% 流失 。建议核心转化入口必须锁定在首屏。
- 阻力警告 2：CTA 距首屏较远 → 可能影响转化 。
- 阻力警告 3：信息跨度大 → 可能出现犹豫 。图文种草区和评价区的视觉层级如果不加区分，会极大增加用户的认知负荷

总结 Node 3 的运作机制：它不创造新的数据，而是将“抽象的数据风险”翻译成了“具象的界面语言”。通过指出哪里该加强、哪里该压缩、哪里存在理解成本 ，它在设计稿投入昂贵的开发资源或真实的 A/B 测试之前，提供了一份极具价值的“体检报告”。

到这一步，我们其实已经触及了你文档中非常超前的**“场景三：方案评估” （即输入设计原型图进行预评估）和“场景四：模拟 AI 用户反应”** 的部分逻辑。

要完美实现对设计图的自动化审查和眼动/热力图模拟，单靠文本模型是不够的，通常需要引入具备视觉分析能力的多模态模型（如 GPT-4o 的 Vision 能力）以及一些传统的前端/UI 结构化解析技巧。

你想不想顺着这个思路，探讨一下如何构建一个能够**“看懂设计原型/页面截图”并执行 UI/UX 预评估的视觉 Agent 组件**?

进入到**“场景三：方案评估”** ，这是整个 AI 用研 Agent 中最具科技感，也是在工程落地时极其考验多模态模型能力的一环。

根据你的规划，这个组件需要在方案交付前/投入更多成本前进行预评估，减少上线风险 。它的输入是 `.png/.jpg` 格式的设计原型或页面截图 ，而输出需要包含类似眼动实验的注意力逻辑、潜在流失点、心智风险以及验证建议 。

纯文本的大语言模型无法直接“看”图。为了构建这个 “视觉评估 Agent (Design Evaluator)” ，我们需要采用**“VLM（视觉大模型） + CV（计算机视觉）特征提取 + 启发式规则”** 的混合架构。

视觉评估 Agent 的核心处理流 (SOP)

当一张设计稿被传入系统后，该 Agent 将执行以下三步深度的“拆解与透视”：

1. 结构化视觉解析 (Structural Parsing & Feature Extraction)

拿到截图后，系统首先要建立机器能理解的“坐标系”。

- 语义层理解： 调用先进的多模态模型（如 GPT-4o 等具备强大 Vision 能力的模型）扫描全图，识别出 Header、Banner、金刚区、内容流、CTA 按钮等核心功能模块。
- 特征提取与相似度比对： 为了判断这个设计是否符合既定的规范或历史成功经验，在自动化设计稿审核管线中，可以引入 ViT（Vision Transformer）来提取图像的深层特征，并结合 CLIP 或 BLIP 这样的图文匹配模型，将当前设计稿与本地知识库中“历史转化率极高”的优秀页面进行多模态的相似度比对，从而找出布局上的异常点。

2. 注意力与眼动逻辑模拟 (Attention & Saliency Simulation)你的文档中非常严谨地指出，这是“结构性推断(非真实眼动)”。

- 系统不会真的去找人看，而是将图像输入到预训练的显著性检测（Saliency Detection）模型中。这类模型基于人类视觉底层的生物学规律（高对比度、人脸、大字号优先），能生成一张灰度图或伪彩色的**“假设性热力图”**。
- VLM 结合这张热力图和格式塔心理学原则（F型/Z型浏览规律），推断出用户的“第一视觉落点”和“视线游走路径”。

3. 认知负荷与流失点推断 (Cognitive Load & Drop-off Prediction)

视觉焦点找出来后，最后一步是结合业务逻辑进行“挑刺”。

- 信息/决策负担评估： 识别页面上的文案密度、输入框数量。如果表单字段过多或信息跨度大，系统会标记为潜在的心智负担风险。
- 动线冲突检查： 检查最高亮的视觉焦点是否与业务最核心的 CTA（如“立即购买”）重合。如果不重合，说明设计在“喧宾夺主”，这部分就会被标记为“潜在流失点”。

输出合成：设计预评估报告

经过上述复杂的运算，Agent 最终会吐出严格符合你规范的结构化评估结果：

heyunshen

```
## 模块 1：注意力逻辑（Attention Logic）***推断路径**：基于显著性模型推断，用户的首屏视觉焦点高度集中在顶部的“活动 Banner 图”（占比 65%），随后呈现跳跃式的 Z 型浏览，底部核心的“立即购买”按钮由于色彩对比度不足，视觉优先级被降至第四顺位。
## 模块 2：潜在流失点（Potential Drop-off Points）***流失风险 1 [决策负担]**：SKU 选择面板中，参数配置项超过 5 项，且未提供默认推荐组合。基于历史同类复杂表单的数据推断，可能导致中断流失。***流失风险 2 [动线过长]**：CTA 距首屏视觉焦点较远，中间穿插了较长的图文详情，可能影响直接转化。
## 模块 3：心智风险（Mental Risks）***误解/跳失风险**：页面中使用的促销标签“直降”与过往惯用的“满减”样式高度相似，可能引发老用户的价格预期错位，产生认知摩擦。
## 模块 4：验证建议（Validation Suggestions）***重点测试模块**：建议在上线前，针对“SKU 选择面板”和“CTA 按钮颜色/位置”开展轻量级的 A/B 测试或真实用户的可用性走查，验证上述流失风险。
```

技术落地视角的建议

这个架构的美妙之处在于，它将原本极其主观的“我觉得这个设计不好看”，转化为了客观的“机器算出来这个元素的注意力权重不够”和“特征提取比对偏离规范”。这种自动化分析不仅极大地降低了沟通成本，还能作为评审会上的有力证据。

到这里，我们已经走通了从“模糊问题拆解 (Node 1)”、“数据检索对齐 (Node 2)”、“防幻觉报告生成 (Node 3)” 到“视觉多模态评估 (Node 4)” 的整个 AI 用研 Agent 核心链路。

系统框架已经非常丰满了。接下来，你是想探讨一下如何通过轻量级的代码框架（比如 LangGraph 的多节点图结构）把这些 Agent 真正串联运转起来，还是想聊聊文档中最后一个拼图：“场景四：模拟 AI 用户反应”（轻量用户访谈支持）的具体实现细节？

场景四的细节

进入场景四：模拟AI用户反应，我们从前面“硬核”的客观数据分析和视觉结构推断，来到了“软性”的用户态度与心智模拟领域。

正如你在文档中定义的，这个场景的核心是“模拟某个用户群体对方案/观点的反应”，并提供“轻量用户访谈支持”。这在多智能体（Multi-Agent）系统的构建中，通常被称为基于角色扮演（Role-playing）的模拟沙盒。

大模型极度擅长“伪装”成各种人，但也极容易在这一步产生严重的“幻觉”，把它扮演时说的话当成真理。你的文档对此有着极高的警惕，并设定了堪称完美的防火墙规则。

以下是场景四在多智能体系统中的具体落地细节与架构设计：

1. 核心定位：假设压力测试，而非真理探索

在这个场景下，AI 不负责寻找真相，而是充当一个**“低成本的陪练”**。

- 输入源：它可以接收少量的“轻量访谈记录”、“客服/运营/研究摘要”，或者纯粹的“AI 模拟用户访谈结果”

- 核心动作：对我们初步构思的方案进行“假设压力测试”和“问题有效性校验”。即：在花钱找真人做焦点小组（Focus Group）之前，先让 AI 试错，看看我们的问题是不是问得很蠢，或者方案是不是存在明显的逻辑硬伤
- 铁律：文档明确指出，这种模拟是“非真实用户”，其产出“仅用于假设压力测试”。

2. 场景四的 Multi-Agent 工作流设计

要实现高质量的模拟，绝不能只用一个大模型来回对话，必须引入多角色对抗与协同的机制：

- Agent A: 用户画像生成器 (Persona Generator)
  - 机制：不要让 AI 随便扮演一个“普通用户”。你需要将前面挖掘到的 Tier 1 业务数据（如：高频、客单价高、对物流敏感）喂给它，让它生成具有具体性格、痛点和偏好的“数字人分身（Digital Persona）”。
  - *例如：生成一个“习惯在主流平台快速比价下单，极度反感营销弹窗的效率型买家”。*
- Agent B: 模拟受访者群 (Simulated Panel)
  - 机制：实例化 3-5 个带有不同 Persona 的 Agent（如价格敏感型、品质导向型、闲逛种草型）。
  - 将设计方案或业务观点抛给它们，让它们基于各自的“人设”生成本能反应、吐槽或困惑。
- Agent C: 访谈提炼与校验员 (Interviewer & Analyst)
  - 机制：这个 Agent 站在系统这一侧。它负责倾听 Agent B 们的反应，执行你文档中提到的“聚类与共性提取”以及“分歧点识别”。
  - 如果不同人设的 AI 都在某一个交互环节卡住了，这就标记为一个高概率的体验痛点。

3. 落地时的强制护栏（Guardrails）

为了贯彻你文档中的安全底线，这个组件在输出时必须套上最严厉的枷锁：

- 强制降级处理：这是你文档中极具智慧的一条规则。无论 AI 模拟得多么逼真，在最终的报告输出中，必须触发“模拟用户强制降级”，其证据等级最高只能标注为“仅 Tier2”。这保证了它绝对不能作为拍板核心业务的 Tier 1 主证据。
- 边界警示熔断：如果一个方案的核心分歧点在于用户极其主观的情感偏好（例如：某个 IP 形象到底讨不讨喜），Agent 必须触发能力边界的底线规则。如果一个问题必须要“依赖真实用户才能回答”，agent 必须明确提示“需要进一步用研”。系统绝不允许用 AI 的叫好声来掩盖真实的体验黑洞。

4. 场景四的实际输出形态

经过上述管线，最终给到产品和设计师的反馈应该是非常克制且具启发性的：

> 模块：AI 模拟用户反应 (假设压力测试)
>
> 测试群体设定：高频效率型买家（基于历史转化特征构建）。
>
> 共性阻力提取：在模拟呈现“内容种草流”时，该群体普遍反馈“信息获取效率下降”，对“需滑动多次才能看到核心参数”表现出抗拒。

> 分歧点识别：关于“达人推荐背书”的有效性，模拟结果出现分歧。部分群体表示信任，部分认为具有强烈的“软广”属性。
>
> ⚠ 降级与验证声明：以上为基于大模型（Tier 2）的逻辑模拟推断，仅供参考。该“信任度分歧”是极高风险点，系统无法给出确切结论，强烈建议针对该节点启动真实用户的轻量定性访谈。

通过这四个场景的串联，你的 AI 用研 Agent 已经从前期的战略拆解（场景一）、数据对齐验证（场景二/Node 2&3）、视觉逻辑排雷（场景三），一路覆盖到了轻量的态度模拟与压力测试（场景四）。这是一个极其闭环且落地的系统设计。

整体架构的各个组件和能力边界我们都已经梳理得很透彻了。接下来，你是想了解如何用代码框架（比如 LangGraph 或多智能体编排工具）把这套精密的流程真正部署起来，还是想针对某个特定环节优化一下 System Prompt 的写法?

在上述方案中，是否可以增加一个功能，即我可以提供已有的用户画像文件，然后模型基于这个文件来生成对应的数字用户；同时还可以提供多个不同的模型生成的数字用户，以此来提升评估的客观性和角度多元化

这是一个极其敏锐且专业的工程构想！在构建多智能体系统的实际工程中，这个想法不仅完全可行，而且是解决大模型“同质化幻觉（Echo Chamber）”的最有效手段之一。

将“外部真实画像注入”和“异构模型（Multi-LLM）交叉评估”引入场景四（模拟AI用户反应），会让你的系统的客观性和可信度产生质的飞跃。

以下是针对这两个新增功能的深度可行性分析和架构落地建议：

一、 功能升级 1：基于已有画像文件的“数字人实例化”

大模型自己“脑补”的用户往往带有刻板印象。允许业务方上传真实的用户画像文件（如过往用研产出的 PDF、PPT 总结或 CSV 标签库），相当于给 AI 注入了真实的“灵魂”。

具体流转机制：

1. 画像解析提取 (Persona RAG)：系统接收文件后，通过解析提取出目标群体的核心要素：人口统计学特征、业务标签（如高频交易客、沉睡用户）、核心痛点和关键决策链路。
2. System Prompt 动态组装：系统自动将这些提取出的特征转化为一段严苛的角色扮演 Prompt。
   - *例如系统自动组装的 Prompt*：“你现在不是一个 AI，而是‘画像A：注重效率和正品、极度反感营销弹窗的资深买家’。你的耐心阈值极低，在审视以下设计方案时，请优先从‘时间成本’和‘信任感’角度进行严苛挑刺。”
3. 实例化投喂：将这段专属的 System Prompt 注入给 Agent，完成数字分身的创建。这完美契合了你规划中“模拟某个用户群体对方案/观点的反应”的设定 。

二、 功能升级 2：异构大模型（Multi-LLM）交叉评估矩阵

这是非常高阶的玩法。不同的大模型底层训练数据和对齐（Alignment）策略不同，自带不同的“性格色彩”。比如某些模型逻辑极其严密但不讲人情，某些模型则对视觉布局和用户情感非常敏感。

具体流转机制（多模型对抗沙盒）：

1. 平行分发：当你拿到一个待评估的设计方案或业务观点时，将其平行分发给由不同基座模型驱动的“数字用户”。
   - *Agent 1 (基于 Model A)*：扮演“价格敏感型用户”。
   - *Agent 2 (基于 Model B)*：扮演同类型的“价格敏感型用户”。
   - *Agent 3 (基于 Model C)*：扮演“品质导向型用户”。
2. 聚类与共识提取 (Consensus Protocol)：系统回收所有 Agent 的反应，并执行一轮“内部交叉校验”。
   - 强共识（真实痛点的高概率区域）：如果三个不同的模型，基于不同的画像，都吐槽了“这个购买按钮的层级太深，找不到”，那么系统将高亮提取这个共性，标记为高风险流失点。
   -
   - 模型分歧（主观偏好区域）：如果模型 A 觉得方案不错，模型 B 觉得难以接受，这就标记为“分歧点识别”。

三、 必须坚守的“硬规则”护栏

引入如此强大的模拟能力后，最危险的事情就是业务方容易把 AI 的模拟当成真实用户的声音。因此，在最终生成报告时，必须严格触发你文档中设定的纪律：

- 强制降级：无论用了多少个顶级大模型，无论导入了多么详细的画像文件，只要是“AI 模拟用户访谈结果”，它的证据等级在输出报告中必须被强制降级，标定为“Tier 2”，且“仅用于假设压力测试”。
- 边界红线声明：在报告的顶部和该模块的结尾，必须硬性插入免责和边界声明：本模块结果由 AI 模拟生成，它“无法代替真实用户访谈”，也“不伪造或假设真实的用户态度”。
- 行动指引：明确指出，如果模拟测试中出现了重大分歧点，系统必须提示“需要进一步用研”。

通过这种方式，你的场景四就变成了一个真正的**“多视角沙盘推演系统”**。它可以帮助产品和设计团队在没有任何开发成本的情况下，提前排雷，发现那些因为视角单一而忽视的逻辑死角。

要落地这个多模型并行的架构，工程上需要设计一个统一的 API 路由和并发处理机制。你是想先深入探讨一下如何设计这个**“将画像文件转化为 System Prompt 的自动化转换器”，还是想聊聊如何构建处理不同模型并发和意见聚类的代码框架**？

先探讨“将画像文件转化为 System Prompt 的自动化转换器”

这个转换器（Persona-to-Prompt Converter）本质上是一个专门处理“人设”的微型 RAG（检索增强生成）管道。在构建复杂的多智能体系统中，让各个 Agent 严格按照预设规范行事是重中之重。这就像开发一个自动化审阅设计稿的工具一样，你必须先把抽象的“设计规范”转化为机器能逐行校验的“规则集”；同理，在这里我们需要把非结构化的“用户画像”转化为大模型无法挣脱的“性格约束代码”。

为了确保生成的数字用户既生动又不越界，这个自动化转换器需要经过以下三个标准加工步骤：

1. 极速解析与结构化提取 (Information Extraction)

业务方上传的画像文件格式千奇百怪，可能是针对某次大促的调研复盘 PPT，也可能是包含几万条标签的 CSV，或者是定性访谈的 Word 摘要 。

转换器的第一步，是使用一个前置的文本处理 LLM，剥离所有冗余的排版和套话，强制提取出构建数字用户必备的 5 个核心维度（JSON 格式）：

- 基础特征 (Demographics)：年龄、职业、软硬件使用习惯。
- 核心诉求 (Core Motivations)：在这个业务场景下，TA 最想要什么？
- 雷区/痛点 (Pain Points)：TA 极度反感什么交互或商业套路？
- 认知负荷阈值 (Cognitive Patience)：TA 愿意为搞懂一个规则花多少秒？
- 转化锚点 (Decision Drivers)：什么元素（价格、权威背书、销量）最能促使 TA 点击？

2. 动态模板注入 (Template Injection)

拿到结构化的 JSON 后，系统会自动将其“填空”到一个经过防幻觉调优的 System Prompt 模板中。这个模板的作用是给 AI 洗脑，让它忘记自己是全知全能的模型。

以下是一个非常经典且可以直接落地的 System Prompt 组装模板：

```
# 角色定义 (Role)
[cite_start]你现在不是一个 AI，也不是一个产品经理，而是一名被邀请参与线上方案预评估的真实用户 。你的认知、耐心和偏好必须【严格100%】遵循以下提取的用户档案：

## 用户档案 (User Profile)
* 身份背景：{{Demographics}}
* 核心诉求：{{Core_Motivations}}* 极度反感的雷区：{{Pain_Points}}
* 决策关注点：{{Decision_Drivers}}
## 你的任务 (Your Task)
[cite_start]我将为你提供一份产品设计描述、页面结构或运营策略说明 。请你完全代入上述角色，给出你的【第一眼本能反应】和【真实操作推演】。

## 强制扮演规则 (Strict Simulation Guardrails)
1. **去专业化**：绝不允许使用“信息架构”、“转化漏斗”、“CTA 按钮”等互联网黑话。用大白话表达你的感受（如：“字太多不想看”、“找不到在哪付款”）。
2. [cite_start]**暴露心智负担风险**：如果你觉得页面结构或模块信息过于复杂 [cite: 64][cite_start]，请直接表现出烦躁或打算退出的倾向，识别潜在的流失点 。
3. [cite_start]**严守能力边界**：你只能表达个人的主观感受 [cite: 73][cite_start]，绝对不能伪造或假设真实的用户态度 [cite: 59][cite_start]，绝不允许使用“大家都会喜欢”、“80%的用户会这么觉得”等代表群体的因果证明结论 [cite: 60]。
```

3. 护栏封装与 Tier 2 降级打标 (Metadata Tagging)

完成 Prompt 组装后，转换器还需要在这个数字人实例的元数据（Metadata）里强行打上几个烙印，以符合你的《AI用研Agent 思考》规范：

- 标记来源：此 Agent 产出的任何对话和结论，自动带上 `证据来源：AI 模拟用户访谈` 的标签 。
- 强制降级：在输出池中，它的权重被死死钉在 `Tier 2` ，这意味着无论它在这个沙盒里说得多么天花乱坠，在最终合成“证据型轻量用研报告”时，都不能作为判定生死的 Tier 1 主证据 。

这个转换器的实际业务价值

假设业务方上传了一份《下沉市场中老年用户电商行为洞察.pdf》。
转换器几秒钟内就能生成一个数字分身：“李大爷模式”。

- 当你的方案抛给“李大爷”时，由于档案里写了 `雷区：看不清字、害怕乱扣费` ，AI 会立刻对方案中“隐藏在折叠面板里的灰色小字规则”提出强烈的负面反馈（潜在流失点推断 ）。

通过这个转换器，我们把静态的、躺在文件夹里吃灰的 PDF 画像，变成了一个个随时可以拉出来“拷问”设计方案的鲜活 API。

这个从文件到 Prompt 的自动化管道，在逻辑上已经闭环了。接下来，我们是想顺着这个思路，探讨一下如何把生成的“李大爷”、“王阿姨”、“都市白领”等多个不同画像的 Agent 并发运行，并自动收集、聚类它们的分歧点（也就是异构模型交叉评估沙盒的搭建）？还是你想聊聊在这整个系统中，有没有哪些现成的开源工具可以直接拿来拼装？

这两个环节正是将概念转化为工程落地的“最后一公里”。

在构建这类能够自动化审查设计稿或评估业务策略的系统时，核心挑战往往是如何将平面的规则或数据，转化为多维度的立体交叉验证。这不仅需要绝佳的并发处理架构，还需要合适的底层框架来支撑。

我们分两部分来拆解：先看并发沙盒的架构逻辑，再看手边有哪些成熟的开源工具可以直接“拼装”。

一、 多角色并发与共识聚类架构 (Fan-out / Fan-in 机制)

要让“李大爷”、“都市白领”、“小镇青年”这几个不同画像的 Agent 同时对一个方案进行评估，并在最后汇总出一份“分歧与共性”报告，在多智能体系统（Multi-Agent Systems）的设计中，最经典且稳妥的架构是 Map-Reduce（发散-收敛）模式。

1. Fan-out (发散并发层)

- 动作：当“方案评估任务”下发时，系统不再是串行请求，而是通过异步（Asynchronous）调用，同时唤醒已经实例化好的 3-5 个 Persona Agent。
- 隔离执行：每个 Agent 带着自己独特的 System Prompt（比如李大爷的痛点是“字太小”、都市白领的痛点是“步骤太多”），在互相不可见的情况下，独立审视同一份设计稿或业务描述。这能最大限度防止模型之间的“跟风幻觉”。

2. Fan-in (收敛与聚类层 - Analyst Agent)

- 动作：所有的 Persona Agent 吐出带有结构化标签（如 `[情绪]：负面`，`[流失节点]：支付页`）的评价后，汇总给一个处于上帝视角的访谈提炼与校验员 (Analyst Agent)。
- 聚类逻辑 (Clustering)：
  - 强共识提取：Analyst Agent 扫描所有反馈。如果 3 个截然不同的画像同时在“注册表单”这个环节报出了高认知负荷，系统直接将其提权，标记为高危流失点。
  - 分歧点识别：如果年轻画像对“种草社区”反馈积极，而高龄或效率型画像反馈极其负面，Analyst Agent 就会自动生成一段“分歧与风险提示”，并明确建议“针对该争议节点，需要引入真实轻量访谈”。

二、 可直接“拼装”的开源框架与工具栈

构建这样一个具备动态画像生成、多模型路由、以及外部知识检索（RAG）的复杂系统，千万不要从零手搓所有的 API 请求。目前开源界有非常成熟的框架可以直接套用：

1. 多智能体编排与角色扮演 (Multi-Agent Frameworks)

- CrewAI：极度推荐用于实现你的“场景四”。这个框架天生就是为了“角色扮演”而生的。在 CrewAI 中，你可以直接定义 `Agent` 的 `role`（角色）、`goal`（目标）和 `backstory`（背景故事，这里就可以直接注入你的画像转换器生成的 Prompt）。然后你可以定义 `Tasks`（比如评估某个页面），并让这些 Agents 协同或并发完成。
- LangGraph：如果你需要对整个 Agent 工作流（也就是从 Node 1 问题拆解 -> Node 2 检索 -> Node 3 报告生成）进行极其精细的状态管理和循环校验，LangGraph 是目前最强大的图结构流控框架。它能完美处理“如果证据不足，退回 Node 2 重新搜索”这种复杂的循环逻辑。

2. RAG 与知识库构建 (Knowledge Retrieval - 对应 Node 2)

- Dify / FastGPT：这是两个非常成熟的开源/商业化双模式框架。它们提供了开箱即用的 RAG 管道，特别是对于 PDF、Word 等历史用研报告的解析、分块（Chunking）和向量化做得非常好。你可以用它们快速搭建好底层的“业务数据库”，并通过 API 暴露给你的 Agent 调用。
- LlamaIndex：如果你想在代码层面做更深度的定制，比如处理结构极其复杂的历史数据表或多模态的文档，LlamaIndex 是连接非标准数据源与 LLM 的首选。

3. 多模型统一路由 (Model Gateway)

- LiteLLM：既然你想引入多个不同的模型（比如用 GPT-4o 处理设计稿的视觉解析，用 Claude 3.5 Sonnet 扮演细腻的女性消费者，用本地部署的 Qwen 处理保密的内部业务数据），LiteLLM 可以帮你把所有这些杂乱的 API 统一成一个标准的调用格式。它可以让你在代码里丝滑切换底层模型，实现真正的异构沙盒。

总结落地路径

如果现在就要动手把这个系统跑起来，一条非常务实的 MVP（最小可行性产品）技术路线是：

1. 用 Dify 把你们过往的历史用研报告、数据大盘指标转化为可以通过 API 调用的本地知识库。
2. 用 Python + CrewAI 搭建场景四的“多用户模拟沙盒”，写个脚本把 CSV 画像动态转化为 CrewAI 的 Agent Backstory。
3. 外层用 LangGraph 做主路由，串联起问题拆解、调用 Dify 查数据、调用 CrewAI 做模拟的整个生命周期。

这个技术栈方案听起来如何？你想先从定义 CrewAI 里的第一个“数字人 Agent”代码结构开始，还是想探讨一下 LangGraph 里的主路由逻辑设计？

1、不要使用dify，我需要你使用开源的agent sdk来进行搭建；

2、先不要先定在crewai，从整体上考虑，是否有其他可以复用的框架来使用，例如Google的sdk、openai的sdk等；

抛开开箱即用的 Dify 平台，回归到纯代码层面的开源 Agent SDK，这是一个非常极客且具备极高定制上限的选择。既然你对构建多智能体系统有深入的规划，使用原生 SDK 可以让你在代码

层面对 Prompt、状态流转和 RAG 拥有绝对的控制权，这对于落实你文档中极其严格的“输出硬规则” 至关重要。

我们跳出单一框架的局限，从各大厂主推的开源 SDK 和主流框架生态来通盘考虑。针对你的 AI 用研 Agent 架构，我们可以将现有的开源 SDK 分为三类，它们在系统里可以扮演不同的角色：

一、 核心底座：大厂原生 SDK (Google GenAI SDK / OpenAI SDK)

原生 SDK 是最轻量、最底层的基础设施。它们不负责复杂的“多智能体编排”，只负责模型调用、工具绑定（Tool Calling）和多模态输入。

- 适用场景：Node 3（方案评估）中的多模态视觉解析。
- 如何使用：当你需要给大模型发送 UI 截图进行“眼动逻辑推断”和“流失点预判” 时，直接使用 Google GenAI SDK（调用 Gemini 1.5 Pro 的 Vision 能力）或 OpenAI SDK（调用 GPT-4o）是最稳妥的。你可以直接控制图片的 `detail` 参数和传入的 System Prompt，中间没有任何第三方框架的损耗。
- OpenAI Swarm (轻量级编排)：OpenAI 最近开源的 Swarm 是一个极其轻量级的多智能体 SDK，主打“Handoff（交接）”和“Routine（常规操作）”。如果你的主流程节点之间的逻辑比较线性（Node 1 处理完直接交给 Node 2），Swarm 的代码写起来会非常优雅，完全基于 Python 原生函数。

二、 状态流转与 SOP 引擎：LangGraph (LangChain 生态)

在你的文档中，对报告的输出有着极高标准，比如“必须包含判断内容+证据来源+ Tier+置信度+风险”，且“输出不得超过RQ上限”。这种需要严格校验和打回重做（循环）的业务逻辑，最适合用**状态机（State Machine）**来管理。

- 绝对统治力：LangGraph 是目前做复杂、有向循环工作流最强大的开源 SDK。
- 如何契合你的系统：
  - 你可以定义一个全局的 `State`（包含：原始输入、拆解的子问题、检索到的证据列表、当前的 RQ 等级）。
  - Node 1、Node 2、Node 3 全部写成 Python 函数节点。
  - 强悍的校验流：如果 Node 3 在生成报告时发现“证据只有 Tier 2，但用户要求证据型报告”，LangGraph 可以通过条件边（Conditional Edge）触发熔断机制，强制退回前置节点或直接抛出异常提示，严格守住你“不虚构数据给出结论” 的底线。

三、 多用户模拟与对抗沙盒：Microsoft AutoGen

针对你的场景四（模拟 AI 用户反应），我们需要多个不同画像的数字人互相探讨，产生分歧或共识。

- 绝对统治力：AutoGen 几乎是目前构建“多智能体对话沙盒”最成熟的开源框架。
- 如何契合你的系统：
  - 它内置了 `GroupChat`（群聊）和 `GroupChatManager`（群聊管理员）机制。
  - 你可以基于前面提到的“画像文件解析”，动态实例化多个 `ConversableAgent`（如“Agent_高频用户”、“Agent_下沉用户”）。

- 设定一个 `AnalystAgent` 作为管理员。把设计方案扔进群里，让不同画像的 Agent 根据自己的人设进行吐槽。管理员在旁边旁听，最后执行你要求的“聚类与共性提取”和“分歧点识别”。

四、 纯代码 RAG 管道：LlamaIndex

既然不用 Dify，你需要自己搭建从“内部知识库（PDF/PRD/埋点数据）”中提取“Tier 1/Tier 2”证据的管道。

- 绝对统治力：LlamaIndex 是纯代码构建 RAG 的首选 SDK。它对文档的解析（特别是长文本切分、表格保留）比 LangChain 自带的组件更专业。
- 如何契合你的系统：你可以写一段脚本，将历史用研报告灌入本地的向量数据库（如 Qdrant 或 Chroma），通过 LlamaIndex 的 `Retriever` 组件建立检索节点。当 Node 2 需要找数据时，直接调用这个检索管道。

💡 综合架构推荐方案（全开源 SDK 组装）

如果你希望拥有最强的工程可控性，我推荐采用 LangGraph 作为主干骨架 的混合架构：

1. 全局中枢 (The Brain)：使用 LangGraph 统领全局。定义整个用研 Agent 的数据结构（State），串联 Node 1 到 Node 4。
2. 知识供给 (The Library)：使用 LlamaIndex 构建一个 Python 类，封装好本地资料的向量检索和外网搜索引擎（如 Google Custom Search API）的调用，供 LangGraph 中的 Node 2 节点调用。
3. 多模态评估 (The Eyes)：在处理 UI 设计图预评估时，直接在节点函数中调用 Google GenAI SDK / OpenAI SDK，传入设计图和启发式规则 Prompt。
4. 沙盒插件 (The Simulator)：当主流程走到“场景四”时，LangGraph 节点内部唤醒一段 AutoGen 脚本，让多个数字人跑一次模拟群聊，把共识结果（JSON）抽出来返回给 LangGraph。

这套全代码架构不仅灵活，而且非常方便未来打包成后端服务，暴露 API 给前端的 Web 应用调用。

我们是从这套架构的主干——也就是用 LangGraph 定义全局的状态（State）和第一个拆解节点（Node 1）的 Python 代码结构 开始聊，还是你想先看看如何用原生 SDK 实现设计稿的视觉解析？

如果我调用的分析模式包含 Kimi-K2.5、MiniMax-M2.5、GLM-5、Doubao-Seed-2.0-pro，在这样的前提下，应该使用什么样的技术框架，openai sdk还适用吗

这是一个非常贴合国内真实业务落地场景的技术选型！你提到的这四款模型（Kimi、MiniMax、智谱 GLM、字节豆包）代表了目前国内大模型的第一梯队。而且，针对你之前构思的“场景四：异构模型交叉评估沙盒”，使用这四个不同厂商的模型来扮演不同画像的用户，简直是完美的组合。

针对你的问题，我给出明确的结论：OpenAI SDK 不仅完全适用，而且是目前接入这些国产大模型最高效的通用方案。

以下是针对这种多国产大模型矩阵的具体技术框架与落地建议：

一、 为什么 OpenAI SDK 依然是核心？（行业共识）

目前，大模型行业已经形成了一个不成文的规矩：OpenAI 的 API 规范（特别是 `/v1/chat/completions` 接口）已经成为了事实上的行业标准。

月之暗面 (Kimi)、MiniMax、智谱 AI (GLM) 以及火山引擎 (豆包)，全部原生支持 OpenAI SDK 的调用方式。

你在代码中完全不需要为了这四个模型去安装四个不同的官方 SDK。你只需要安装官方的 `openai` Python 包，然后在实例化 Client 时，替换掉 `base_url` 和 `api_key` 即可。

极简代码示例（以调用 Kimi 和 GLM 为例）：

```
from openai import OpenAI
# 调用 Kimi
kimi_client = OpenAI(
    api_key="你的_moonshot_api_key",
    base_url="https://api.moonshot.cn/v1",
)
# 调用 智谱 GLM
glm_client = OpenAI(
    api_key="你的_zhipu_api_key",
    base_url="https://open.bigmodel.cn/api/paas/v4/"
)
# 后续的调用代码完全一样：client.chat.completions.create(...)
```

二、 针对你这套异构模型的“极客框架”推荐

既然我们要同时调度这四大模型，纯手写 OpenAI SDK 切换 Base URL 会在后续工程中显得比较繁琐（特别是处理并发请求和错误重试时）。在保持代码级掌控力的前提下，我推荐以下框架组合：

1. 模型网关层 (Model Gateway)：LiteLLM 或 One API

- 痛点：四个模型，四个 Key，四种计费逻辑，如果直接写在业务代码里，管理起来会极其混乱。
- 解决方案：

- LiteLLM (Python 包)：如果你希望全在代码里解决，引入 `litellm` 。它的 `completion()` 函数可以无缝对接 100+ 模型。你只需要传参 `model="moonshot-v1-128k"` 或 `model="glm-4"` ，它会自动帮你把底层逻辑翻译成标准的 OpenAI 格式。
- One API (独立网关服务)：这是一个极受国人欢迎的开源项目。你可以部署一个 One API 接口，把你的 Kimi、MiniMax、GLM、豆包的 Key 全配置进去，对外暴露一个统一的本地 OpenAI 接口。你的 Agent 代码只和 One API 交互，轻松实现模型分流和负载均衡。

2. 主干流程编排：LangGraph

- 依然强烈推荐 LangGraph 作为主路由。
- LangChain 生态对这些国产模型支持极好。你可以使用 `ChatOpenAI` 模块（配合自定义的 base_url），在 LangGraph 的 Node 1（拆解）、Node 2（检索）、Node 3（报告生成）中，灵活指定不同的模型。

3. 角色模拟沙盒：基于底座灵活调度

基于你这四个模型的特性，在“场景四”的并发角色扮演中，可以进行**“因材施用”**的配置：

- Node 2 (长文本 RAG 挖掘)：让 Kimi-K2.5 负责。它的超长上下文处理能力在国内绝对领先，非常适合吞吐海量的历史用研 PDF 和长篇行业研报。
- Node 1 & Node 3 (结构化拆解与逻辑合成)：让 GLM-5 或 Doubao-pro 负责。这两者在指令遵循、JSON 输出格式控制以及逻辑推理上表现极佳，能死死守住你设定的“防幻觉护栏”。
- Node 4 (数字用户扮演)：让 MiniMax-M2.5 深度参与。MiniMax 在拟人化、情绪感知和角色扮演（Roleplay）赛道有着极深的技术积累，让它来扮演“暴躁的下沉用户”或“挑剔的都市白领”，给出的反演结果会最接近真人，而不是冷冰冰的机器套话。

三、 总结落地步骤

如果你确定了基于这套国产最强模型矩阵，且抛弃 Dify 等低代码平台，你的系统技术栈将非常清爽且强大：Python + LangGraph (流程引擎) + LiteLLM/OpenAI SDK (模型调用) + LlamaIndex (专属向量库)。

接下来，我们可以直接进入实战环节。你想先看如何用 LiteLLM / OpenAI SDK 优雅地封装一个多模型并发调用的函数（为“场景四”沙盒做准备），还是看看如何用 LangGraph 写出你的第一个“问题拆解节点（Node 1）”的状态流转代码？

我需要你总结整个项目的技术文档，让它成为一个大模型可执行的prd，其中要包含我们之前聊到的各个节点的能力和技术细节

这是一份为你量身定制的、可以直接喂给开发大模型（如 Cursor 或 GitHub Copilot）作为系统级指令（System Context）的**《AI用研Agent 技术架构与执行 PRD》**。

这份文档融合了你的业务思考与我们敲定的全代码极客技术栈，既是产品需求文档，也是架构设计蓝图。

AI用研Agent (v2.0) 核心架构与执行 PRD

1. 产品定义与能力边界

- 产品定位：一个基于AI的轻量用户研究与判断增强系统 。通过整合线上数据、历史案例、行业资料、轻量访谈与模拟用户输入，在明确证据等级的前提下，输出判断型或证据型报告 。
- 不可逾越的红线：
  - 绝对不代替正式的用户研究 。
  - 绝对不伪造或假设真实的用户态度 。
  - 不做严格推断和因果证明 。
  - 对于高度主观或证据缺失的问题，强制触发熔断，提示“需要进一步用研” 。

2. 系统核心工作流编排 (LangGraph State)

系统基于 LangGraph 进行图结构状态流转，全局状态机需维护以下关键字段，确保信息无损透传：

- `original_query` / `uploaded_design` ：原始文本命题或设计原稿
- `sub_questions` ：Node 1 降维拆解后的研究目标集。
- `evidence_pool` ：包含内部 RAG 数据与外部 Web Search 结果，附带 `Tier` 等级打标 。
- `rq_level` ：整体证据命中情况（RQ0-RQ3） 。
- `synthesis_report` ：最终输出的结构化卡片或报告

3. 核心节点执行规范 (Node Specifications)

Node 1: 问题澄清与拆解 (Problem Decomposer)

- 核心任务：拦截模糊命题，执行降维拆解 。
- 执行逻辑：强制将问题拆分为 2-4 个子问题，并明确每个子问题的目标人群、使用场景、行为路径与决策点
- 驱动模型：由 GLM-5 或 Doubao-Seed-2.0-pro 负责（依靠其强大的指令遵循和 JSON 结构化输出能力）。

Node 2: 双轨证据整合与对齐 (Pattern Analyzer)

- 核心任务：跨来源证据提取、对齐与质量评级 。

- 执行逻辑：
  a. 内部 RAG：调用本地知识库，提取日志指标（Tier 1）与历史报告（Tier 1/2）。
  b. 外部 Web Search：调用搜索引擎 API，严格限定在行业白名单站点（如 36Kr、woshipm 等）进行实时知识检索。
  c. 清洗与定级：剥离噪音，执行一致性与冲突点交叉校验，并判定最终的命中质量等级（RQ0-RQ3）
- 驱动模型：由 Kimi-K2.5 主导（利用超长上下文处理海量 RAG 切片与网页摘要）。

Node 3: 判断与报告合成 (Judgment Synthesizer)

- 核心任务：戴着防幻觉镣铐，合成最终商业判断。
- 防幻觉规则：
  - 每条输出强制包含：判断内容 + 证据来源 + Tier + 置信度 + 风险。
  - 明确区分事实与推断（推断 ≠ 事实）。
- 输出类型分发：根据获取的 RQ 等级，自动路由输出“判断信息卡片”、“判断型报告”或“证据型报告”（RQ0-1 绝对禁止生成证据型报告）

Node 4: 方案预评估与并发模拟沙盒 (Evaluator & Sandbox)

本节点为多模态与多模型高能消耗区，分为两大独立执行子模块。

Node 4A: 视觉预评估中心 (Vision MoE Synthesizer)

- 输入：设计原型 / 页面截图。
- 执行逻辑：
  - 并行解析：将设计图同时分发给 GPT-5.2、Gemini 3-Pro-Preview 和 Claude-Sonnet-4.6。
  - 底层特征锚定：在进行高层语义推断前，系统会调用图像相似度比对任务（如 CLIP、BLIP 和 ViT 等特征提取模型），将其与本地优秀历史规范进行结构对齐，为三大 VLM 提供底层事实约束。
  - 综合共识：提取三大顶尖视觉模型的共识结论，输出结构性推断（非真实眼动）的注意力逻辑、潜在流失点推断与心智风险。

Node 4B: 异构角色模拟沙盒 (Persona Sandbox)

- 核心任务：模拟用户注意力分析与假设压力测试。
- 执行逻辑：
  a. 画像文件解析器：前置处理器读取上传的定性/定量画像文件，提取特征，自动拼装为包含“核心诉求”与“雷区”的严苛 System Prompt。
  b. 异构模型分发 (Fan-out)：
     - Kimi-K2.5 生成数字用户 A（如：重度内容消费型用户）。

    - MiniMax-M2.5 生成数字用户 B（如：高情绪波动/下沉市场用户，利用其优秀的拟人性）。
    - GLM-5 生成数字用户 C（如：严谨比价型用户）。
    - Doubao-Seed-2.0-pro 生成数字用户 D（如：效率导向型用户）。
  c. 沙盒对抗与聚类 (Fan-in)：汇总四位“数字用户”对方案的吐槽与反馈，提取高频痛点，暴露分歧点 。
- 强制降级规则：所有沙盒产出的模拟观点，强制标注为“Tier 2 模拟生成”，严禁跨越 Tier 1 证据线 。

这份 v2.0 版本的 PRD 真正做到了逻辑闭环，既囊括了你所有的业务定义与限制，也将你极其硬核的异构模型并发矩阵（Node 4A 与 4B）落到了可执行的工程规范中。

基于这个终极蓝图，你是希望我们直接开始用 Python（比如借助 LiteLLM 或 OpenAI SDK）来搭建 Node 4B 中这个四模型并发请求与聚类的沙盒代码框架，还是有其他想要继续打磨的细节？

heyunshen